

Data Mining - Graph Spectra

Theodor Popescu, Samouil Mosios

December 1, 2025

1 Instructions To Access Datasets

The two graph datasets we used as part of this report are the two example .dat files presented as part of the homework assignment on Canvas.

2 Introduction

In this seminar, we implemented the spectral clustering algorithm described in the paper by Andrew Y. Ng, Michael I. Jordan, and Yair Weiss in MatLab, and used this algorithm to analyze clusters within two different graphs and draw conclusions about those two graphs.

3 Implementation

Our implementation follows the proposed algorithm from the paper very similarly, except for the creation of the affinity matrix. They define the affinity matrix according to the a similarity norm between two points, however we do not have any attributes to our nodes, as our data only includes the standard graph edge data. As a result, we used the adjacency matrix A as an equivalent to the affinity matrix described in the paper for our use case:

```
As= sparse(col1, col2, 1, max_ids, max_ids);  
A = full(As);
```

We then construct the diagonal matrix D from A, and use this along with A to create our normalized Laplacian matrix $L = D^{-1/2} * A * D^{-1/2}$:

```
D = diag(sum(A, 2)); %Diagonal Degree Matrix
```

```
D_inv_sqrt = diag(1 ./ sqrt(diag(D)));  
L = D_inv_sqrt * A * D_inv_sqrt; %Normalized Laplacian;
```

Once the normalized Laplacian L is created, we can find the k largest eigenvectors of L , and create a new eigenvector matrix X by having the columns of X be the k largest eigenvectors. We choose k by how many clusters we want in our spectral clustering.

```
k = 2; % Cluster Number
[X, eigenValues] = eigs(L, k, 'largestabs'); %Extracting k largest eigenvalues
```

We then do a re-normalization of X so that each row has unit length:

```
for i = 1:N
    row_norm = sqrt(sum(X(i, :).^2)); % compute the norm of each row
    Y(i, :) = X(i, :) / row_norm; % divide each element by row norm
end
```

And finally, once we have the normalized eigenvector matrix, we perform a simple k means clustering algorithm on it (with our specified k from before) to compute the final cluster assignments for each node in the graph:

```
[labels] = kmeans(Y, k, 'Replicates', 10); %Run k means on the normalized matrix
```

4 Methods for Evaluation

We evaluated how our spectral clustering performed in multiple ways. Firstly, we graph our nodes and all respective edges, identifying each node by their assigned cluster. This allows us to visually see the different clusters in our graph, and allows us to identify clear patterns in our graph should they exist, and as a result we can see how well our clusters are assigned. Secondly, we compute some cluster metrics to gauge how well our clusters are performing, mainly each cluster's average internal degree (how many edges within the cluster a node of that cluster has), each cluster's average inter-cluster degree (how many edges outside the cluster a node of that cluster has), and the overall averages among all clusters. These numbers can give us a good representative idea of how well the clusters are representing our graph: as one of the trademarks of accurate clusters is a high average internal degree (cluster similarity), and a low average inter-cluster degree (cross-cluster interaction). Finally, we also plot the sorted Fiedler vector (second smallest eigenvector of our Laplacian matrix) as that gives a visualization of the communities present in our graph, and so we can use this to cross-reference our findings.

5 Graph Evaluations

5.1 Graph 1

The first graph we evaluate is a dataset on medical advancements in four different towns in Illinois. Given this knowledge about the graph having 4

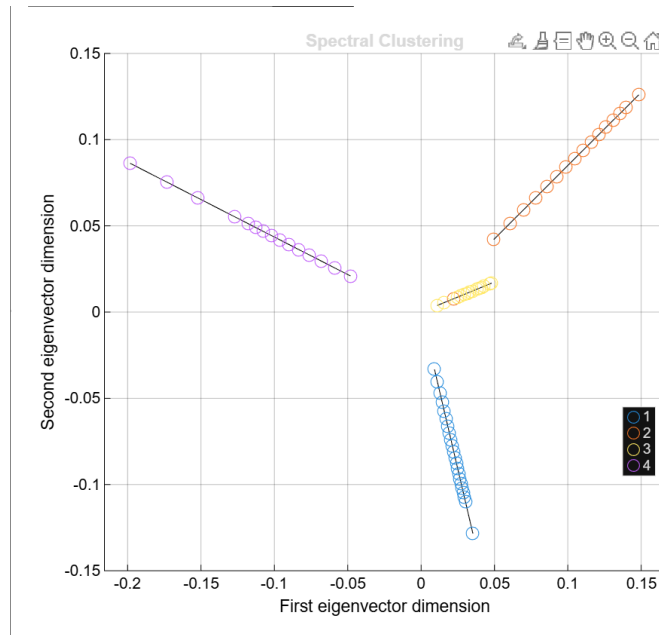
different towns, a first thought would be to cluster the graph into 4 clusters. Additionally, when we construct the normalized Laplacian, we can also construct the basic graph Laplacian ($L = D - A$), which has the property that the number of eigenvalues = 0, corresponds to the number of connected components in the graph:

```
GL = D - A; % construct normal Laplacian
```

```
e = eig(GL); % get all eigenvalues
```

```
num_zero_eigenvalues = sum(abs(e) < 1e-10); % count eigenvalues close to 0
```

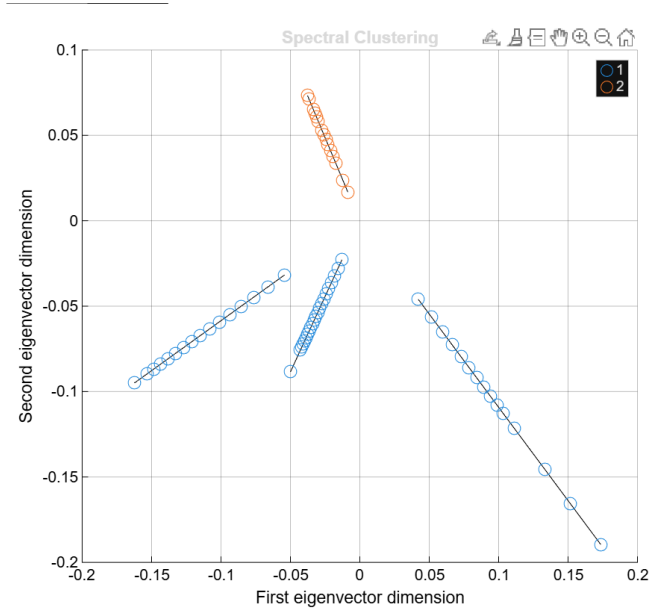
For this graph, the graph Laplacian has 4 eigenvalues, which confirms that there are 4 distinct connected components in our graph that correlate to the 4 towns the dataset was done in, which again points to 4 clusters, one per component, as being an optimal representation. When running our algorithm with 4 clusters, the graphed results seem to confirm this, as our 4 clusters seem to line up extremely well with the 4 towns represented on the graph:



We can see that the 4 clusters generated are very accurate to the graph's structure, and the cluster metrics confirm this, with an average internal cluster degree of around 9 and an average inter-cluster degree of less than 0.2:

Num of Clusters	Average Internal Degree	Average Inter-Cluster Degree
2	9.4	0.0
3	9.04	0.0
4	9.05	0.17
6	7.89	0.89
8	7.27	1.52

We can see that once we start to introduce more clusters than our graph's structure supports, we start to increase the average inter-cluster degree as communities of nodes that are naturally connected and have edges to each other begin to get sorted into separate clusters. We can also see that having 2 or 3 clusters does seem to give some minimal improvement over having 4 clusters, however when visualizing the actual graph with 2 clusters and 3 clusters, we notice that the natural communities in the graph just get combined to form the larger clusters:

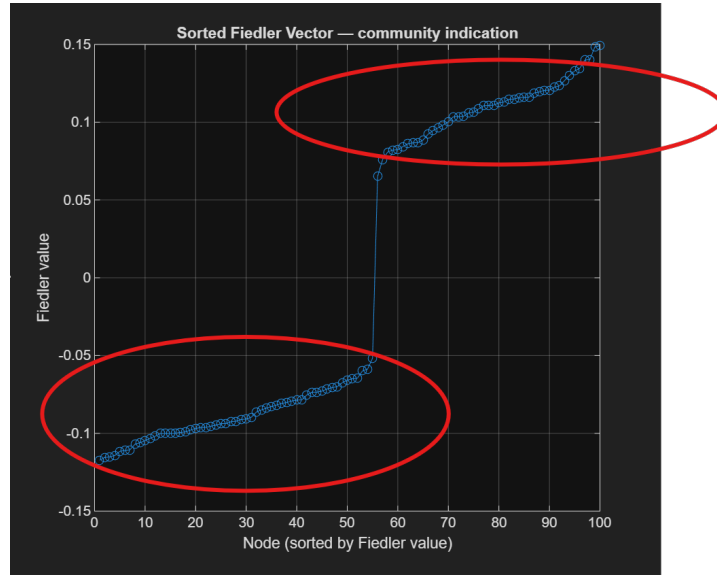


Given that our graph does not have any node attributes to form similarity measures on, this cluster merging doesn't harm our graph metrics, but we can see visually that it combines separate connected components in our graph. This causes a representation that we can see does not match our graph nearly as accurately as a 4 cluster structure would, and causes the clusters itself to be less cohesive and have unrelated components grouped together. We can see that therefore both from our intuition of the graph itself, and from the results from our spectral clustering algorithm, that 4 is the correct amount of clusters to partition this graph dataset into.

5.2 Graph 2

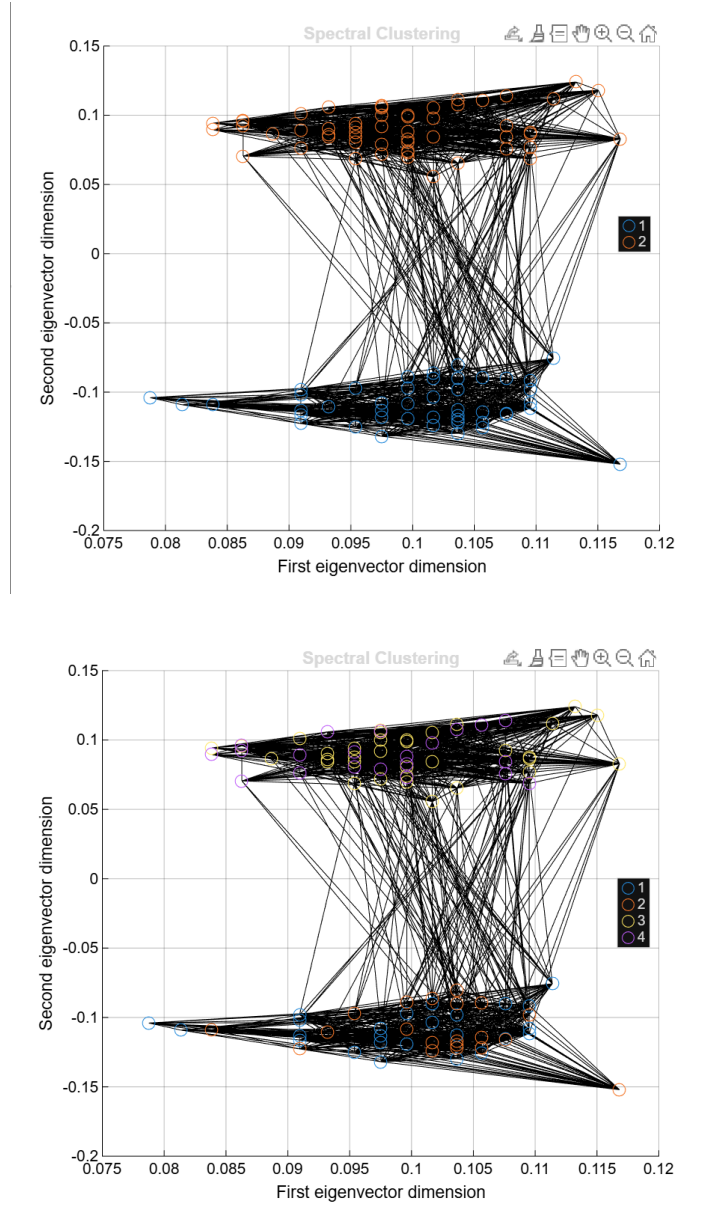
The second dataset is a synthetic graph where we do not have any information regarding how it was formulated. When checking the number of 0-eigenvalues for the graph Laplacian matrix, we get that there is only 1 (for the trivial eigenvector) - which means the graph is made up of only one connected component.

This doesn't give much initial information, but we can gain additional context by plotting the sorted Fiedler vector values. The Fiedler vector corresponds to the second lowest eigenvector of the graph Laplacian matrix, and plotting the sorted Fiedler Vector can show communities within the graph. For this graph data, the plotted Fiedler vector seems to show two natural communities:



Analyzing the cluster metrics across different numbers of clusters, as well as observing our graph clusters visually, further confirms that clustering into two is the most accurate for this graph:

Num of Clusters	Average Internal Degree	Average Inter-Cluster Degree
2	23.93	2.28
3	12.98	11.08
4	8.50	15.67
6	5.63	18.46
8	3.76	20.36



We can see that with 2 clusters, we have a very high average internal degree, along with a very low inter-cluster degree. Increasing the number of clusters at all causes the internal degree to be heavily reduced, and the inter-cluster degree to increase significantly, which shows a far lower cluster representation accuracy with any cluster number higher than 2.

Additionally, we can see visually that our graph data seems to correlate our findings from the sorted Fiedler vector and our cluster metrics, as we can clearly observe two heavily packed communities in our graph, with

much more sparse connections between those communities. Our clustering of two seems to seamlessly categorize our points into these two communities, whereas having a higher cluster number forces these dense communities to be split into separate clusters. This split causes edges that were previously captured within the cluster to now be inter-cluster, and given the relatively dense communities this split occurs in, causes the large shift observed in the average internal and inter-cluster degrees.

From the initial sorted Fiedler vector and graph cluster visualizations, as well as the observations of our cluster metrics, this graph seems to have by far the best representation when clustered into 2.

6 Takeaways and Conclusion

From implementing our own spectral clustering algorithm, we were able to see how different methods can be used to analyze graph structures, and to accurately cluster graphs into an optimal number of clusters. This has far reaching implications, as this is not only useful for graph data, but also for other types of datasets that have some attributes/measure of similarity. There are many situations where it is useful to be able to group data in to certain buckets, and being able to deduce the optimal number of buckets in order to maximize the similarity inside a bucket/minimize the similarity across buckets as well as partition the data into that optimal number of buckets is extremely important.